

```

> # 加分題 A
> aa <- read.delim("SRBCT_train.txt") # 讀檔
> bb <- as.matrix(aa[,-c(1,2)]) # 扣掉第 1、2 col，因為是 character
>
> genes_name <- aa[, 1] # 取得基因名稱
> # 進一步依「樣本層級的標準差 (sample-wise sd)」對樣本排序，完成以下：
>
> # 1.用 order() 找出 sd 最大的前 5 個樣本。
> samples_sd <- apply(bb, 2, sd)
> top_5_samples_sd <- order(samples_sd, decreasing = TRUE)[1:5]
> top_5_samples_sd
[1] 58 63 46 10 13
>
> # 2.用 rank() 算出每個樣本的 sd 排名。
> rank_samples_sd <- rank(samples_sd)
EWS.T1 EWS.T2 EWS.T3 EWS.T4 EWS.T6 EWS.T7 EWS.T9 EWS.T11 EWS.T12 EWS.T13 EWS.T14 EW
S.T15 EWS.T19 EWS.C8 EWS.C3 EWS.C2 EWS.C4
      14      13      15      55      10      19      6      36      49      60      54      58
      59      51      30      31      1
EWS.C6 EWS.C9 EWS.C7 EWS.C1 EWS.C11 EWS.C10 BL.C5 BL.C6 BL.C7 BL.C8 BL.C1 B
L.C2 BL.C3 BL.C4 NB.C1 NB.C2 NB.C3
      5      18      39      43      27      47      26      20      34      40      41      46
      17      24      16      11      50
NB.C6 NB.C12 NB.C7 NB.C4 NB.C5 NB.C10 NB.C11 NB.C9 NB.C8 RMS.C4 RMS.C3 RM
S.C9 RMS.C2 RMS.C5 RMS.C6 RMS.C7 RMS.C8
      33      38      22      9      23      2      28      21      45      57      7      61
      56      29      8      35      4
RMS.C10 RMS.C11 RMS.T1 RMS.T4 RMS.T2 RMS.T6 RMS.T7 RMS.T8 RMS.T5 RMS.T3 RMS.T10 RM
S.T11
      48      3      32      25      12      52      63      44      42      37      53      62
> # 3.檢查 order() 得到的前 5 名，是否與 rank() 的結果一致。
> N <- length(rank_samples_sd)
>
> ## 因為 rank 是從小到大，且沒辦法用 decreasing，所以前五就是後五個值
> top5_rank_samples_sd <- which(rank_samples_sd > N - 5)
EWS.T13 EWS.T19 RMS.C9 RMS.T7 RMS.T11
      10      13      46      58      63

```

R沒有一個名字就叫 `set` 的專屬資料型態,但可以用幾種方式達成 Python `set()` 的功能(去重複、集合運算)。

最接近的做法:用向量 + `unique()`

R的慣用做法是用普通向量,再搭配集合相關函數。去除重複值就用 `unique()` :

```
r
x <- c(1, 2, 2, 3, 3, 3)
unique(x)      # 1 2 3
```

這相當於 Python 的 `set([1,2,2,3,3,3])`。差別在於 R 的向量有順序、允許重複,只是你透過 `unique()` 去重,而 Python 的 set 本身就不允許重複、無序。

集合運算函數

R base 內建了對應數學集合運算的函數,直接作用在向量上:

```
r
a <- c(1, 2, 3, 4)
b <- c(3, 4, 5, 6)

union(a, b)      # 聯集: 1 2 3 4 5 6
intersect(a, b)  # 交集: 3 4
setdiff(a, b)    # 差集(a 有 b 沒有): 1 2
is.element(3, a) # 成員測試: TRUE,等同 3 %in% a
```

`%in%` 是很常用的成員判斷運算子:

```
r
3 %in% a          # TRUE
c(3, 7) %in% a    # TRUE FALSE (可向量化判斷多個)
```

判斷兩集合是否相等

R沒有直接的 set 相等運算子,常見寫法是:

```
r
setequal(a, b)    # 忽略順序與重複,判斷是否為同一集合
```

```
> ## AI
> setequal(top_5_samples_sd, top5_rank_samples_sd)
[1] TRUE
```

```

> # 4.對 sd 最大的那個樣本，找出其表現值最高的前 10 個基因。
> max_sd_sample <- bb[, top_5_samples_sd[1]]
> top10_genes_max_sample_sd <- order(max_sd_sample, decreasing = TRUE)[1:10]
> top10_genes_max_sample_sd
[1] 1781 1774 735 1574 509 1572 1834 1975 551 292
>
> # 5.做出一張 summary table，包含下列欄位：
> # 樣本名稱、sample-wise sd、sd 排名，以及該樣本前 10 高表現基因的 Image Id 與表現值。
>
> ## 存放每個樣本前 10 高表現基因的值
> top10_gene_expression <- array(vector("list", 63), dim = c(63, 1))
>
> ## 存放每個樣本前 10 高表現基因的 Image Id
> top10_gene_id <- array(vector("list", 63), dim = c(63, 1))
>
> # 用迴圈跑每一個 col
> for (i in 1:63){
+   top10_gene_expression[[i]] <- sort(bb[, i], decreasing = TRUE)[1:10]
+   idx <- order(bb[, i], decreasing = TRUE)[1:10] # 找到前十的原始 idx
+   top10_gene_id[[i]] <- genes_name[idx] # 找到並存放 id
+ }
>
> table <- data.frame("樣本名稱" = colnames(bb), "sample-wise sd" = samples_sd,
+   "sd 排名" = rank(samples_sd),
+   "Top 10 Image Id" = top10_gene_id,
+   "Top 10 Gene Expression" = top10_gene_expression,
+   row.names = NULL)

```

蔡耀斌_hw_1.R		蔡耀斌_hw_1_加分.R		table	
Filter					
	樣本名稱	sample.wise.sd	sd.排名	Top.10.Image.Id	Top.10.Gene.Expression
1	EWS.T1	0.9023934	14	c("530814", "1492412", "470261", "884546", "85 [...]	c("4.2609", "4.2452", "4.2285", "4.2222", "4.1434" [...]
2	EWS.T2	0.8952745	13	c("51293", "970591", "878798", "773479", "770704" [...]	c("7.6184", "7.1260", "6.5933", "6.3725", "6.3383" [...]
3	EWS.T3	0.9059050	15	c("33525", "51293", "878798", "853570", "470261" [...]	c("5.5842", "5.5745", "5.4433", "5.3632", "5.3489" [...]
4	EWS.T4	1.1925464	55	c("842784", "768299", "789091", "51293", "147 [...]	c("10.4754", "10.1145", "9.5846", "9.5706", "9. [...]
5	EWS.T6	0.8663332	10	c("47833", "34396", "811028", "51448", "248295" [...]	c("6.1149", "5.5302", "5.4848", "5.2707", "5.1655" [...]
6	EWS.T7	0.9343484	19	c("43733", "31143", "843139", "809578", "470261" [...]	c("7.1461", "6.7403", "6.3422", "6.1931", "6.1767" [...]
7	EWS.T9	0.8483575	6	c("31143", "33525", "1475730", "470261", "23 [...]	c("7.7609", "7.4448", "7.3220", "7.0526", "6.9105" [...]
8	EWS.T11	1.0391300	36	c("43550", "52076", "33525", "34849", "773479" [...]	c("7.6902", "7.6821", "7.6172", "7.5128", "7.2301" [...]
9	EWS.T12	1.1529887	49	c("52076", "43733", "377461", "773479", "3 [...]	c("8.8089", "8.5556", "8.2508", "8.2480", "8.1499" [...]
10	EWS.T13	1.3418554	60	c("122159", "51293", "842784", "214990", "34849" [...]	c("19.2768", "18.1841", "15.2291", "13.5094", "11. [...]
11	EWS.T14	1.1775772	54	c("842784", "809578", "853570", "35185", "97 [...]	c("10.0654", "9.9905", "9.7157", "9.1315", "8. [...]
12	EWS.T15	1.2658881	58	c("842784", "857681", "470261", "34849", "88 [...]	c("13.4761", "13.1366", "12.7845", "12.6878", "11. [...]
13	EWS.T19	1.2677206	59	c("773479", "35185", "34849", "842784", "22493" [...]	c("18.3899", "18.2642", "15.2652", "14.7822", "11. [...]
14	EWS.C8	1.1612246	51	c("1435862", "51293", "884842", "809578", "5 [...]	c("7.4097", "7.3593", "7.1051", "7.0916", "6.7212" [...]
15	EWS.C3	0.9871340	30	c("970591", "796646", "1476065", "83011", "4 [...]	c("5.5391", "5.4348", "5.3506", "5.3203", "5.2884" [...]
16	EWS.C2	0.9954100	31	c("796646", "40017", "970591", "208718", "627939" [...]	c("6.6230", "6.4549", "6.0808", "6.0212", "6.0058" [...]
17	EWS.C4	0.7455796	1	c("970591", "842973", "884546", "292424", "511521" [...]	c("3.2980", "3.2864", "3.2363", "3.1959", "3.1875" [...]
18	EWS.C6	0.8318380	5	c("34849", "770394", "1461138", "897567", "77 [...]	c("4.1015", "4.0513", "3.8451", "3.8089", "3.7992" [...]
19	EWS.C9	0.9232795	18	c("770394", "34849", "1461138", "898262", "3 [...]	c("5.2958", "5.0101", "4.9875", "4.8199", "4.7639" [...]
20	EWS.C7	1.0691206	39	c("34849", "1435862", "42352", "51293", "146 [...]	c("6.8416", "6.7491", "6.7358", "6.6345", "6.4651" [...]
21	EWS.C1	1.0979321	43	c("33525", "32257", "878676", "796646", "77 [...]	c("6.7607", "6.7202", "6.6926", "6.6703", "6.6624" [...]
22	EWS.C11	0.9725914	27	c("773479", "731308", "1435862", "51293", "4 [...]	c("5.6366", "5.6310", "5.5589", "5.5560", "5.4918" [...]
23	EWS.C10	1.1268339	47	c("1472643", "949939", "47833", "950482", "3 [...]	c("6.6813", "6.6638", "6.4991", "6.4118", "6.4056" [...]
24	BL.C5	0.9649714	26	c("194384", "34849", "853570", "51293", "88 [...]	c("6.4247", "6.1513", "6.0293", "5.9637", "5.8804" [...]
25	BL.C6	0.9442416	20	c("1486260", "51293", "194384", "868304", "4 [...]	c("5.8352", "5.6840", "5.5418", "5.4986", "5.4334" [...]
26	BL.C7	1.0255234	34	c("884842", "33525", "842784", "853570", "840942" [...]	c("6.4711", "6.3473", "6.2815", "6.2766", "6.2027" [...]
27	BL.C8	1.0811283	40	c("194384", "1472643", "1471829", "33525", "86 [...]	c("6.8174", "6.4483", "6.3997", "6.3606", "6.3424" [...]
28	BL.C1	1.0854579	41	c("884546", "1476065", "1492412", "796646", "4 [...]	c("6.0407", "5.9480", "5.8616", "5.8150", "5.7184" [...]
29	BL.C2	1.1259233	46	c("1476065", "949971", "884546", "970591", "88 [...]	c("8.0634", "7.6601", "7.6257", "7.4565", "7.3092" [...]
30	BL.C3	0.9222812	17	c("970591", "884546", "1492412", "33525", "147 [...]	c("4.6113", "4.5059", "4.4993", "4.3359", "4.3056" [...]
31	BL.C4	0.9586906	24	c("83011", "970591", "1492412", "884546", "4 [...]	c("5.6856", "5.4857", "5.4660", "5.4241", "5.4184" [...]
32	NB.C1	0.9186860	16	c("810552", "758365", "884842", "770704", "87 [...]	c("5.2446", "5.0271", "4.8721", "4.8682", "4.7224" [...]
33	NB.C2	0.8783569	11	c("1461138", "40017", "82225", "486787", "4 [...]	c("4.5958", "4.5633", "4.5235", "4.4736", "4.3557" [...]
34	NB.C3	1.1563298	50	c("884546", "970591", "296448", "868308", "84 [...]	c("9.0924", "9.0157", "8.8936", "8.3042", "8.2528" [...]
35	NB.C6	1.0061901	33	c("773479", "47833", "842784", "34357", "35185" [...]	c("6.3747", "6.2091", "6.1919", "5.9042", "5.8691" [...]
36	NB.C12	1.0522149	38	c("32257", "884842", "345833", "882510", "882484" [...]	c("6.6530", "6.6115", "6.2075", "6.1171", "6.1089" [...]
37	NB.C7	0.9551821	22	c("884842", "1492147", "786680", "878578", "84 [...]	c("5.5604", "5.3127", "5.2531", "5.2481", "5.2242" [...]
38	NB.C4	0.8653178	9	c("51293", "34849", "47833", "35185", "842784" [...]	c("5.1894", "5.1835", "5.1750", "4.9352", "4.9072" [...]
39	NB.C5	0.9557501	23	c("51293", "450777", "194384", "44563", "853570" [...]	c("5.4701", "5.3549", "5.1733", "5.1184", "5.1165" [...]
40	NB.C10	0.7951259	2	c("884719", "27549", "42096", "47833", "882510" [...]	c("4.3224", "4.3171", "4.2346", "4.2201", "4.1969" [...]

> # 加分題 B

> # 用兩種方式選「高變異」基因：

> # 主題目的「排序取前 30%」

```
> genes_sd <- apply(bb, 1, sd)
```

```
> n_top30_cols <- ceiling(nrow(bb) * 0.3)
```

```
> top30_genes_sd <- order(genes_sd, decreasing = TRUE)[1:n_top30_cols]
```

>

> # 用 `quantile(genes_sd, 0.7)` 當門檻，取高於門檻者

```
> th = quantile(genes_sd, 0.7)
```

```
> top30_sd = genes_sd[genes_sd >= th]
```

```
n_top30_cols <- ceiling(nrow(bb) * 0.3)
top30_genes_sd <- order(genes_sd, decreasing = TRUE)[1:n_top30_cols]
th = quantile(sd_genes, 0.7)
top30_sd = sd_genes[sd_genes >= th]
用which()回報這份資料下兩法各選出幾個。
```

Top30 個高變異基因

比較兩種基因篩選方法的計數差異。

兩個方法的選取邏輯不同，所以 `which()` 回報的數量也會不同。先假設 `genes_sd` 和 `sd_genes` 是同一份資料(你的程式碼裡兩個名字不一致，我當作同一個向量處理)。

方法一：取排名前 n 個(固定數量)

```
r
n_top30_cols <- ceiling(nrow(bb) * 0.3)
top30_genes_sd <- order(genes_sd, decreasing = TRUE)[1:n_top30_cols]

# 用 which() 回報數量
sel1 <- rep(FALSE, length(genes_sd))
sel1[top30_genes_sd] <- TRUE
length(which(sel1)) # = n_top30_cols
```

這個方法一定會選出 `ceiling(nrow(bb) * 0.3)` 個，因為它是照排名硬取前 n 個。所以答案就等於 `n_top30_cols`，`which()` 在這裡只是驗證。

方法二：取 $sd \geq 70$ 百分位(門檻篩選)

```
r
th <- quantile(sd_genes, 0.7)
length(which(sd_genes >= th)) # 實際選出的數量
```

這個方法用門檻 `>=` 去篩，數量不保證剛好是 30%，原因有二：

一是 `quantile()` 預設 `type = 7` 會做內插，門檻值本身可能落在兩個資料點之間；二是若剛好有多個基因的 `sd` 等於(或非常接近)門檻值，`>=` 會把這些平手的全部納入，導致選出的數量比 30% 略多。

一次比對兩者

```
r
# 方法一
n1 <- n_top30_cols
# 方法二
n2 <- length(which(sd_genes >= quantile(sd_genes, 0.7)))

cat("方法一選出:", n1, "個\n")
cat("方法二選出:", n2, "個\n")
cat("差異:", n2 - n1, "個\n")
```

實務上通常 `n2 >= n1`。如果你希望兩法選出的數量一致，方法二可以改用嚴格大於 `>` 或改變 `quantile()` 的 `type`，但最穩定的做法還是方法一那種「照排名取固定 n 個」。

把你的 `bb` 資料實際跑一下這段，就能得到這份資料下的兩個確切數字了。

🔍 🔍 🔍 🔍 🔍

```
> # 用 which() 回報這份資料下兩法各選出幾個。
> ## AI
> n_top30_sd <- length(which(genes_sd >= th))
> n_top30_genes_sd_simple <- length(top30_genes_sd)
> # 比較兩者是否一致
> n_top30_genes_sd_simple == n_top30_sd
[1] TRUE
>
> # -----
> # 加分題 C
```

abline()各參數代表的意義

`abline()` 是 R base 繪圖裡加「直線」的函數,通常在 `plot()` 之後呼叫,把線疊加到現有圖上。它的參數可以分成幾組,對應不同的畫線方式:

主要參數(決定畫哪種線)

`a`, `b`:用截距(intercept)和斜率(slope)畫線,對應 $y = a + bx$ 。例如 `abline(a = 2, b = 3)` 畫出截距 2、斜率 3 的斜線。

`h`:畫水平線,值是 y 座標。 `abline(h = 5)` 在 $y = 5$ 畫一條橫線。可以給向量一次畫多條,如 `abline(h = c(1, 2, 3))`。

`v`:畫垂直線,值是 x 座標。 `abline(v = 0)` 在 $x = 0$ 畫一條直線。同樣可給向量。

`coef`:直接給一個長度為 2 的向量 `c(截距, 斜率)`,等同於同時指定 `a` 和 `b`。例如 `abline(coef = c(2, 3))`。

`reg`:傳入一個迴歸模型物件(通常是 `lm()` 的結果),自動抓出係數畫回歸線。這是最常用的用法之一:

```
r
fit <- lm(y ~ x)
plot(x, y)
abline(reg = fit)      # 或直接寫 abline(fit)
```

外觀參數(控制線的樣式)

這些其實是從 `par()` 繼承來的通用圖形參數:

`col`:線的顏色,如 `col = "red"` 或 `col = "blue"`。

`lty`:線型(line type),`1` 是實線、`2` 是虛線、`3` 是點線,依此類推,也可用名稱如 `"dashed"`。

`lwd`:線寬(line width),數字越大越粗,預設 `1`。

幾個常見組合範例

```
r
plot(x, y)
abline(h = 0, v = 0, col = "gray", lty = 2)  # 畫灰色虛線的座標軸
abline(lm(y ~ x), col = "red", lwd = 2)      # 紅色粗迴歸線
abline(a = 0, b = 1, lty = 3)                # 對角參考線 y = x
```

```
> # 用 hist() 畫出全部 gene 的 sd 分布,並用 abline(v = ...) 標出 top 30% 的切點。
> ## AI
> hist(genes_sd, nclass = length(genes_sd))
> abline(v = th, col = "red", lwd = 2, lty = 2)
```